Gasballoon Cockpit

A single-file HTML web app for gas balloon flight planning and in-flight monitoring: live position tracking, wind-based landing predictions, staged descent planning, weather stations, air traffic, and offline map caching - built for use on a tablet in the basket.

Current version: v260824.07-0720 (24.08.2026) - this number always matches the `APP_VERSION` constant near the top of the script in `index.html`. Versioning scheme: `vYYMMDD.zz-HHMM` (date of the last change, a 2-digit counter that resets to 01 each new day, and the build time), so multiple same-day builds are unambiguous at a glance - the version is shown in the bottom-left corner of the app itself, useful for confirming a deployment actually picked up the latest build rather than a stale cached one. `cors_test.html`'s own version marker is kept in sync with this. `admin.html` is versioned independently.

What it is

One HTML file, no build step, no server. Open it in a browser (or install it to a home screen as a PWA, with `apple-touch-icon.png`) and it runs entirely client-side, calling a handful of free public weather/mapping APIs directly from the browser. A service worker (`sw.js`) caches map *tile* requests only - deliberately not weather/elevation data, which needs to stay fresh - so previously-viewed map areas remain available if the connection drops.

Password gate on load (SHA-256 hashed, set in `GATE_HASH`).

Built for genuine 24/7 unattended operation: the service worker checks for its own updates every 30 minutes (not just on page reload, which might never happen once the app is left running), and a screen wake lock keeps the display on with automatic re-acquisition if the browser ever releases it.

The hamburger menu's last entry, "About - more info!", opens this README as a PDF in a small viewer page with always-visible Print/Close buttons (readme_viewer.html), rather than the browser's native PDF viewer directly.

Zoom-in/out buttons (and the very first automatic centring on GPS after boot) target the visually VISIBLE map area's centre, not the underlying map container's own geometric centre - relevant whenever the sidebar is open, since it overlays on top of the map rather than shrinking it.

Main functions

The app has two main functions, switched via a pill toggle at the top of the map.

Landing Area (default)

Shows the balloon's live predicted trajectory: a cruise segment (green) that transitions into a descent (yellow) at a configurable time ("Initiate descent in"), continuously recalculated as position, altitude, and wind data update. A Monte Carlo scatter of likely landing points is shown as a shaded polygon around the predicted landing point, with a black cross marking its actual centre (computed from the raw scatter itself, not the containing polygon's own vertices, which would skew it toward the edge). The map view gently follows this polygon as it shifts

in response to slider changes - only actually panning/zooming if it would otherwise fall outside the visible area, debounced so dragging a slider doesn't cause the map to jump on every intermediate value.

The descent point (a yellow cross on the trajectory) can be adjusted two ways that stay in sync: the "Initiate descent in" slider (10-minute steps, defaults to 20min), or dragging the cross itself along the trajectory.

Extended trajectory preview: click anywhere within roughly 60° of the trajectory's own direction, beyond where the normal prediction/landing area already reaches, to see the trajectory extended further out (blue) - a frozen snapshot from the moment of the click, useful for comparing the real flown track against an earlier prediction.

A one-time onboarding hint appears the first time this function is used.

Plan Descent

Two sub-functions, switched via a smaller toggle directly under the main "Plan Descent" button.

Quick Descent (default): click any point on the map to find the nearest reachable landing area and the descent path to it - shown as an orange cross (the calculated descent point) and a yellow descent path, same visual language as Landing Area's own live prediction. The reference trajectory truncates at the descent point once a plan is active. The descent point can be dragged to explore alternatives, or a new point clicked anywhere to restart. The reference trajectory itself extends as far as the "Initiate descent in" slider is set to.

Staged Descent: drag a royal-blue cross marker along the trajectory to choose when descent begins, then click within the resulting reachable area to search for a staged descent plan - one that pauses at one or more intermediate altitudes (chosen automatically where wind conditions genuinely differ) rather than descending continuously. Opens a side panel with a chart of the planned stages: altitude, wind at each level, duration, an estimated ballast requirement for each transition, and markers for any wind shear, inversion, or isothermal layers encountered. Minimum stage separation and maximum intermediate stages live directly in this panel. Its own two sliders (max descent time - 10-minute steps, defaults to 20min; inter-stage rate) sit side by side in one row.

Both sub-functions show a one-time onboarding hint on first use. Switching back to Landing Area clears the descent path from the map but leaves the reachable area, Monte-Carlo landing area, and its centre marker in place, with a "Clear staged descent area" button to remove them explicitly.

Flight data box

A small draggable panel showing course (always 3 digits, e.g. 065°), speed (km/h and kn), climb/sink rate, and altitude (m AMSL, ft, flight level above a configurable transition altitude, and AGL). Refreshes on a fixed 0.5s cadence, decoupled from the underlying GPS fix rate. Snaps flush against whichever map edge it's dragged past; defaults to the left edge just below the header button row. Dims along with the map in night mode.

Descent Parameters

"Initiate descent in" and "Descent rate" sit side by side. Descent rate's own track is a wedge shape that grows taller toward higher values, visualizing the accelerating descent speed - the draggable point itself still runs along a flat baseline; only the fill behind it

changes shape. Below each slider: the computed distance/effective rate, plus (for Descent rate specifically) the adiabatic lift bonus as a weight-equivalent ballast figure - the actual extra lift the adiabatic cooling effect provides during descent, typically a few kg. Intercept AGL, Rate below, and Scatter sit in a second row, each with its own computed time-to-reach badge.

Landing Area sidebar card

Shown for whichever landing-point marker is currently the relevant one (Landing Area's live prediction, Quick Descent's planned point, or Staged Descent's), with these fields:

- **Ground wind** at the predicted landing time, with wind gusts (when the model supports them) shown as a second line below it.
- **Terrain near landing point**: elevation-based roughness (flat / moderate slope / steep), nearby tagged buildings or forest, the specific land cover at the exact point (meadow, farmland, forest, residential, water, etc.), and a red "CAUTION WATER!" override if the point falls inside a mapped lake, river, or reservoir - checked via genuine polygon containment, not just proximity. This check runs on its own independent request queue, separate from the app's other Overpass-based features, so a failure elsewhere (airports, roads, place names) can't block it.
- **Sun at estimated landing**: azimuth (always 3 digits) and elevation, with a direction arrow that rotates to point at the sun's actual position, plus remaining time until evening civil twilight (HH:MM) - the more relevant cutoff than sunset itself for whether there's still enough light to see the terrain.
- **Moon phase**: a filled SVG silhouette (not an emoji, for consistent rendering across devices) with illumination percentage and a small waxing/waning trend arrow, plus rise/set times.

Weather modeling

Wind is fetched per-altitude across 19 pressure levels (18 for the two Météo-France models below, which don't publish 975hPa) from the auto-selected model for the current location (ICON-D2 for DACH, ICON-EU for wider Europe, GFS globally), refreshed on a timer (default 15min, configurable). CAPE and wind gusts are fetched as separate, independent requests on a reduced schedule (every 3rd wind refresh) so an unsupported variable on a given model can never take down the core wind fetch.

Besides the auto-selected models, ARPEGE Europe (Météo-France, 11km, all of Europe) and AROME France (Météo-France, 2.5km, France and immediate neighbours) can be picked manually for model comparison - both publish the pressure-level data this app needs. MeteoSwiss's own ICON-CH1/CH2 (1-2km, Switzerland) and the UK Met Office's models were deliberately not added: as of this writing neither publishes pressure-level/wind-at-altitude data through Open-Meteo, which the wind profile this app relies on requires.

Ensemble modeling: with a commercial Open-Meteo API key set (see below), the Monte-Carlo landing-area scatter in all three functions draws on real ensemble model members (ICON-D2-EPS / ICON-EU-EPS / GEFS, ~20-40 members depending on model) instead of an artificial age-based heuristic - each Monte-Carlo sample uses an actual member's own wind profile, genuine forecast uncertainty rather than a random perturbation. A small badge next to the model name in the header ("E" green / "H" gray/amber) shows which mode is currently active. Falls back to the heuristic automatically without a key or if the ensemble fetch fails. Open-Meteo's Ensemble API specifically requires the Professional or Enterprise commercial tier - a Standard/base commercial key (which unlocks the main forecast/elevation/CAPE endpoints just fine) is not enough; the badge

turns amber rather than plain gray in that specific case, distinguishing “no key at all” from “key present but this tier doesn’t include ensemble access”. The account behind this app’s key was upgraded to API Professional on 17.08.2026, confirmed reachable via a live test (HTTP 200). With the resulting higher monthly quota (5M vs 1M calls), the ensemble fetch’s own pressure-level set was also widened from 8 to 11 levels for finer vertical resolution.

Also found and fixed once real data was reachable: Open-Meteo’s ensemble member keys use 2-digit zero-padded numbering (“_member01”, not “_member1”) - the code previously reconstructed keys without that padding, so members 1 through 9 (a majority of a typical 20-40 member ensemble) silently found no matching data and ended up with empty wind profiles, which windAt() then treats as zero wind. This would have severely distorted the scatter even once the API itself became reachable. The padding width is now detected directly from a live response’s own keys rather than assumed.

Power lines

Overhead line towers and transformers are rendered as small, muted markers rather than bright coloured circles, so they don’t visually compete with the lines themselves (able to be followed continuously) - spotting the wires is what actually matters for flight safety, not each individual mast along the route. Towers (neutral grey) and transformers (muted blue-grey) use distinct tones, not identical ones, so toggling one category’s checkbox in Settings while leaving the other on stays visually verifiable rather than looking like filtering stopped working. Category checkboxes and the “overhead only” toggle now discard and fully rebuild the power-lines layer instance from scratch (not just remove/re-add the same one) - a first fix relied on VectorGrid’s own redraw() (documented upstream as unreliable), a second removed and re-added the same instance, and even that turned out insufficient, very likely because VectorGrid keeps its own internal tile cache tied to the instance itself regardless of map attachment. Rebuilding the instance entirely cannot carry over any stale internal state.

Found the actual source of persistent “blue circles and half-circles not connected by any line” that survived all three earlier fix attempts above: OpenInfraMap’s combined tile endpoint bundles several vector layers beyond power - telecoms (including microwave radio links, point-to-point wireless connections with no cable between them, often drawn as isolated markers or directional half-circle antenna-orientation symbols), petroleum, and water infrastructure. Only the power_* layers were ever explicitly styled; any other layer name fell back to VectorGrid’s own default rendering, which is what those blue shapes almost certainly were. The whole style definition is now wrapped in a Proxy so every layer name NOT explicitly listed is hidden by default - robust against not knowing (or OpenInfraMap changing) the exact non-power layer names, rather than needing to enumerate and hide each one individually.

Telecom/antenna masts are still shown, though - unlike the invisible microwave links, a physical mast is exactly as real a hazard for a balloon as a power tower, so a dedicated “Telecom/antenna masts” category (distinct magenta colour, own Settings checkbox) covers several plausible layer-name candidates at once, since the exact one couldn’t be confirmed from public documentation. A one-time console log now also records every layer name actually encountered that isn’t explicitly styled, to nail down the real name(s) directly from a live tile rather than guessing further.

Also found while working on this: the power-layer legend’s Tower and Transformer swatches still showed their original bright colours (yellow-orange, light blue) from before those categories were muted - the legend was never updated alongside that actual styling change, so it was silently showing the wrong colours since then. Now corrected

to match. Separately, restoring saved category selections from a stored profile now also refreshes the legend to match - it previously only updated the layer itself, so the legend could keep showing every category regardless of what had actually been saved as deselected.

Staged Descent

The Monte-Carlo landing-area scatter (run after a specific stage plan is found for a clicked target) uses antithetic sampling - each random draw is simulated together with its exact opposite - rather than fully independent draws. The wind uncertainty here is a constant bias for an entire simulated run that accumulates over the whole (potentially hours-long) path rather than averaging out step-by-step, so a small independent sample could land asymmetrically by chance and pull the drawn landing area's centroid noticeably away from the deterministic best-found plan (and so away from the intended target) rather than sitting close to it, as had been reported. Antithetic pairing guarantees the scatter stays symmetric around that deterministic path regardless of sample size.

License

Custom, source-available license in `LICENSE` (Wicki Aero GmbH) - permits running/hosting an unmodified copy, but not modification or redistribution without prior written permission. See that file for the full terms and third-party component licenses.

Airspace

The ✈ airspace layer draws real, class-filtered airspace boundaries from openAIP's Core API (confirmed working via a live CORS test - it was previously believed CORS-blocked) on top of the combined raster tile as a visual base (still the only way to show airports/navaids/reporting points, since openAIP retired separate per-category tile endpoints in 2023). The Settings checkboxes (Class A-G, Restricted, Prohibited, TMZ, RMZ) now genuinely filter which boundaries are drawn, refetched as the map moves or the selection changes. An airspace whose type/class isn't recognized by the mapping used here is shown regardless of checkbox state, rather than silently hidden.

Commercial Open-Meteo API key: optional field in Settings. When set, every Open-Meteo request in the app (wind, CAPE, gusts, elevation, ensemble) automatically switches to the dedicated `customer-api.open-meteo.com` endpoint, which has no daily call limit, instead of the free tier's 10,000/day cap - needed for genuine 24/7 unattended operation, since a single wind-profile request already counts as several calls toward that quota (any request covering more than 10 variables does). A running "Open-Meteo calls today" counter is shown in the weather model popover regardless of which tier is active.

Data sources

Source	Used for	
Open-Meteo	Wind profiles, elevation, CAPE, wind gusts, precipitation, ensemble members	Working. O removes the
SondeHub	Radiosonde launches, for using real sounding data instead of the	Working

	model	
api.existenz.ch (SwissMetNet)	Swiss weather stations	Working
Bright Sky	German weather stations	Working
MeteoGate/E-SOH	Weather stations across the rest of Europe (33 countries)	Station list parsing not live respons
ADSBExchange (via RapidAPI) + OGN/glidernet.org	Air traffic, deduplicated where both sources report the same aircraft	Working
RainViewer	Rain radar	Working
Overpass API	Airspace, terrain/land-cover, roads, region names, airports (4 mirror fallbacks: overpass-api.de, overpass.private.coffee, lz4.overpass-api.de, maps.mail.ru)	Working. Th own indepe share a sep one can't bl limited mirr excluded fr would silen elements" f coverage, ri terrain resu retriable fai
Nominatim	Reverse geocoding for landing-area place names	Working
openAIP	Airspace tiles	Working
MetarCentral	Airport METARs	Working. Ai list of majo falling back (any OSM-t ICAO code)
aprs.fi	APRS station tracking	**Not functi** the browse Settings cle
Xweather	Lightning (only polls when rain is detected nearby)	Confirmed strikes retu `success:tru` as describe parameter to 5 minute without the on.

		Confirmed
		Business St
		`current/{lat`
		`forecast/{la`
		real data, w
		(any coordi
		registered l
		in this tier -
		so this is a
		not a wind-
		full field set
		dewpoint, h
		pressureMs
		snowAmou
		windSpeed/
		weatherSyn
		"cloudy"/"p
		station-leve
		individually
		(with the sp
		ANALYSIS

Meteologix/Kachelmannwetter	Independent ground-conditions cross-check at the landing point, plus precipitation probability	station). A c endpoint, a `tools/weath` looked like lookup tabl many differ found (all a failure) - wc the confirm `station/{st` path, and fc small self-c matched to (robust aga yet). Now a markers on (toggled toc METAR/Swi identities o point locati points arou position are `current`, and response ac deduplicate not tied to r only every 2 per-move re adds up fas Starter tier triggered o

Settings

Organized into color-coded groups: General (cache radius, transition altitude, units), Weather (model selection, sonde data, METAR/station sources, lightning), Air Traffic & Airspace, Terrain & Ground Features, Descent Planning (adiabatic model), Safety & Positioning (external GPS, emergency contact), and Data & Backup.

Tokens and keys: every external service credential the app uses, gathered in one place and individually editable - Open-Meteo (commercial), openAIP, RapidAPI (air traffic), aprs.fi, Xweather, EmailJS. As with any client-side app, these are visible in the page source to anyone who views it; changing one asks for confirmation first.

Profile export/import: as a JSON file (native share sheet on mobile) - independent of the shared flight-profile system below, useful for a one-off transfer between devices without needing the shared store at all.

Login and shared flight profiles (replaces the old Dropbox/encrypted-Gist backup system, 24.08.2026)

The single fixed password gate was replaced with a 3-letter user code + password login, validated against a shared user table stored in a GitHub Gist, accessed via a Cloudflare Worker proxy (`WORKER_URL`/`WORKER_APP_SECRET` constants near the top of the script - see the dedicated section below for why). A session stays valid for 12h before the gate reappears.

The same login screen also offers opening an existing **flight profile** (an 8-character name + 6-digit PIN) - a full settings snapshot shared across all users who know the PIN, stored in the same Gist. Opening one loads its settings immediately; a “New profile” button on the login screen proceeds without loading one (stays in local-only mode until explicitly saved as a named profile later). The currently active profile is shown in the footer, next to the version number (“Flightprofile: XXXX” or “(local)” when none is active).

Closing Settings after changing anything prompts to save: to the currently-open flight profile (re-asks its PIN first), to a brand-new named profile (asks for a new name + PIN, rejecting an already-taken name), or only locally (the existing `localStorage`-based behavior, unchanged).

AdminApp (`admin.html`, deployed alongside `index.html`): a separate, simple, master-password-protected page for managing the shared Gist directly - two editable tables (users: code+password; flight profiles: name+PIN, both shown in plain text, since none of it is sensitive), a “Create new Gist” button for first-time setup, and its own version number. The Gist ID/token are entered here by whoever administers it (kept only in that browser’s own `localStorage`, not hardcoded in this file) - once a Gist exists, its ID and a `gist`-scoped token get pasted into `index.html`’s own constants so the main app can read/write it too.

Emergency contact: pilot name, aircraft registration, mobile number, and email, with prepared-message sending over WhatsApp, SMS, or email (via mailto, or silently via EmailJS if configured).

Trajectory modeling fixes

Found the actual, deeper cause of Quick Descent’s reported 180° reversal (17.08.2026), after an earlier fix (capping the delay search at 240 minutes, described below) turned out insufficient on its own: every simulation step resolved which hour’s forecast wind to use via `Math.round(t/3600)` rather than `Math.floor(t/3600)`. Hourly forecast data represents the full hour starting at that mark - so 30 minutes into a flight is still within hour 0’s forecast, not already hour 1’s - but `round()` was switching to the next hour’s forecast a full 30 minutes too early. If that next hour’s forecast differs meaningfully from the current one (a realistic case, not a data error), the shown path would appear to snap onto a different wind direction partway through, exactly matching what was reported. This affected every trajectory simulation in the app (Landing Area, Quick Descent, Staged Descent, and the “wind at landing site” readouts) - all 19 occurrences of this rounding pattern are now consistently `floor()`.

Separately, fixed a genuinely broken Staged Descent: its reference trajectory (the line the staged-descent marker is dragged along) had its length tied to the “Initiate descent in” slider for consistency with Quick Descent - reasonable while that slider defaulted to 120 minutes, but once its default was later reduced to 20 minutes (in an unrelated change), the reference line became far too short to drag a marker on, search for a plan within, or show any reachable area/Monte-Carlo scatter at all. Staged Descent’s reference trajectory is now a fixed 240 minutes regardless of that slider, independent of Quick Descent’s own horizon.

Also fixed the earlier-described bug: Quick Descent’s search for the descent-initiation delay that lands closest to a clicked target could pick a delay of up to 600 minutes (10 hours) if that happened to land closer, whenever the normal 240-minute range still looked “improving” at its edge - an operationally unrealistic delay for an actual flight. The search is now kept within the same 0-240 minute range the “Initiate descent in” slider itself offers.

Found and fixed a further contributor to nonsensical-looking paths, particularly visible when testing without a real GPS fix: the first 1-3 minutes of every simulated cruise phase blended FROM the observed course/speed toward the forecast wind - but without a real fix yet (courseKnown still false), state.course/speedKn sit at arbitrary placeholder defaults (90°, 15kn), not an actual observation. Blending from that placeholder was pulling the very start of every path toward that arbitrary direction regardless of the real wind. With no genuine observation to blend from, the simulation now uses the actual forecast wind from t=0 instead in that situation.

Also fixed a confusing display inconsistency: the “Initiate descent in” label next to the slider only ever showed the slider’s own raw value, never updating to reflect what a Quick Descent search actually found - so it could show e.g. “20 min” right next to an “Estimated descent point” reading of “12.1 min” for the very same plan, two numbers for what looks like the same thing. The label now shows the plan’s actual found value (marked “(plan)”) while a Quick Descent plan is active, and reverts to the slider’s own value once the plan is cleared - without changing the slider’s own position, which remains Function 1’s independent setting.

Cruise-phase vector hierarchy (per explicit specification, 17.08.2026): as long as a real GPS course/speed is currently known, that vector has priority for the ENTIRE cruise-phase extrapolation - the reference trajectory is the straight-line continuation of the presently observed course/speed, for as long as that observation is available. Only when there is no GPS signal (which can also happen temporarily mid-flight, not just before the first fix) does the trajectory fall back to the forecast wind model at the current altitude instead. This replaces an earlier approach that blended from observed course toward forecast wind over just 1-3 minutes regardless of whether GPS remained available throughout - which meant even a long, fully GPS-tracked cruise phase would end up governed by the wind forecast instead of the actual observed movement almost the entire time.

Minimum-speed refinement to that hierarchy (21.08.2026, threshold adjusted the same day per feedback that 8km/h was too high): found that GPS course-over-ground, derived from successive position fixes, becomes dominated by GPS position jitter rather than genuine displacement at low ground speed - a basic, well-established GPS limitation. A near-stationary or slow-moving receiver can report a technically-real course reading that’s essentially noise, unrelated to actual heading - live-reported as the trajectory pointing NNW on a ~3.6kn GPS course reading while strong wind aloft was blowing SW. Below 2kn (~3.7km/h) reported ground speed, GPS course is no longer trusted for the cruise-phase extrapolation even when technically known, falling back to the forecast wind model instead - consistent with the already-handled “no GPS at all” case.

Wind particle height slider ceiling now stays current (23.08.2026): the slider’s maximum (current altitude AGL + a fixed buffer, so it doesn’t uselessly extend far past what’s actually relevant) was previously only computed once, at the moment the ground-wind layer was first switched on - left running for the rest of a flight (the normal case), that ceiling stayed frozen at whatever the balloon’s altitude happened to be at that moment, eventually preventing the slider from reaching the balloon’s actual current altitude at all as it kept climbing. Now recalculated on every recompute cycle instead, the same way the live landing area itself already was. Buffer set to a fixed 1750m above current AGL altitude, always, per explicit instruction the same day.

Staged Descent search quality: found (in the code’s own prior comment) that the reachable-area grid resolution had been deliberately reduced in an earlier fix, reasoning that a finer grid was what made double-clicks feel unreliable - but the actual reentrancy guard already protects against that independently of resolution, so the reduction only traded away search quality without fixing anything. Too

coarse a grid gave the subsequent refinement step a poor starting point, converging on a mediocre local optimum well short of the actual clicked target - producing a landing-area scatter clustered far from the target, with only the deliberately-added target vertex stretching the drawn polygon out to a thin triangle shape (matching what was reported). Resolution raised back to a genuinely fine grid.

Staged Descent reachable-area reliability: the same reentrancy guard, on a closer look, discarded an overlapping recomputation request outright rather than queuing it - since several different triggers (marker drag, various sliders) can each request a recomputation within a short window, silently dropping one left the displayed reachable area stale relative to the most recent actual state, which is very likely why it stopped showing reliably. It now remembers that another run was requested and automatically re-runs once, immediately after the current one finishes.

Quick Descent's visible fork/kink at the path's end: found the actual cause (17.08.2026) - the descent path's own last point was being deliberately overwritten with the Monte-Carlo landing area's centroid, on the reasoning that the path should visually reach the landing-area centre marker exactly. But that centroid is a statistical average over many separate, uncertainty-perturbed simulations, not a point the single deterministic path shown actually reaches - forcibly snapping the final segment onto it produced a visible kink/fork right at the end whenever the centroid sat off to the side of where the path was actually heading, rather than directly ahead of it. The path now ends at its own real, physically-simulated endpoint; the landing-area centre marker (already drawn separately) continues to show where the Monte-Carlo centre actually is.

Quick Descent's visible fork/kink at the path's end: found the actual cause (17.08.2026) - the descent path's own last point was being deliberately overwritten with the Monte-Carlo landing area's centroid, on the reasoning that the path should visually reach the landing-area centre marker exactly. But that centroid is a statistical average over many separate, uncertainty-perturbed simulations, not a point the single deterministic path shown actually reaches - forcibly snapping the final segment onto it produced a visible kink/fork right at the end whenever the centroid sat off to the side of where the path was actually heading, rather than directly ahead of it. The path now ends at its own real, physically-simulated endpoint; the landing-area centre marker (already drawn separately) continues to show where the Monte-Carlo centre actually is.

Quick Descent's wrong zoom target / seemingly-missing landing area: found that Function 1's own "keep the landing area visible" logic (`ensureLandingAreaVisible`, debounced 500ms) kept running even while Plan Descent (Quick/Staged) was active, and would then overwrite the zoom Quick Descent had just correctly set (onto its own Monte-Carlo hull) with a zoom based on Function 1's own, unrelated data near the current position - pushing the actual Quick Descent landing area outside the visible map area, which looked exactly like "the landing area isn't shown" even though it had been drawn correctly. Now only acts while Function 1 (Landing Area) is actually the active function.

Staged Descent reference trajectory now stops at the marker: the green reference line previously always extended out to its own fixed length (240 minutes) regardless of where the staged-descent marker (the blue cross) had actually been placed - it's now cut off exactly at the marker, the same way Quick Descent's own reference line is cut at its chosen descent point, since nothing beyond the marker is actually part of how the balloon gets there.

Staged Descent search: refining from multiple starting points: coordinate-descent refinement previously polished only the single best grid candidate, which can settle into a mediocre local optimum well short of the actual clicked target if the true optimum sits in a different

region of the strategy space than that one grid point. Now independently refines the top 4 grid candidates and keeps whichever converges closest to the target - aimed at the reported “landing area stuck in one corner, shaped like a thin triangle” pattern, which is consistent with the search converging on a plan that doesn’t actually land near the target, leaving the Monte-Carlo scatter clustered far from it with only the deliberately-added target vertex stretching the drawn shape out to a point.

Root cause found for several seemingly-unrelated “landing area not showing” reports (17.08.2026): `state.appMode` was never explicitly initialized - it stayed `undefined` from boot until the first mode-switch click, even though Landing Area is genuinely the default active function shown on startup. Every `state.appMode==='monitor'` check elsewhere in the app (including the fix described just above, which keeps the view-following logic from fighting with Plan Descent’s own zoom) evaluated false during that entire initial period as a result - so the map’s own “keep the landing area visible” logic never actually ran on a fresh load, leaving the correctly-computed Monte-Carlo area and its centre marker outside the initial zoomed-in view around the current position, which looked exactly like “the landing area isn’t shown” even though it had been drawn correctly. Now explicitly initialized to `'monitor'`.

Staged Descent’s landing area not settling around the target: found the actual, structural cause - the drawn landing-area polygon always included the clicked target as a forced extra vertex, specifically to “guarantee” it stayed inside the polygon regardless of how well the search converged. Whenever the actual best-found plan didn’t land exactly on the target (routine, since the reachable strategy space is discrete/constrained), that forced vertex stretched the polygon into a wedge/triangle shape with the target at one tip and the real scatter clustered at the other - never honestly centred AROUND the target the way a genuine uncertainty cloud should be. This forced vertex is now removed entirely; the landing area shows only the natural Monte-Carlo scatter around the actual best-found plan, which - combined with the multi-start refinement described above - should now settle close to the target on its own merits rather than being artificially stretched to reach it.

Quick Descent’s zoom sometimes never actually happening: found the real cause after a closer re-check (17.08.2026) - the zoom-to-landing-area call sat AFTER two network-dependent operations (updating the landing position card, fetching a wind profile for the descent point), neither wrapped in error handling. A failure in either one (a terrain-check or geocoding request timing out, for instance - not rare, per the Overpass 429/504s seen in testing) silently aborted the whole function before the zoom call was ever reached, even though the landing area itself had already been drawn correctly just before that point. The zoom call is now placed immediately after the landing area is computed, before those network-dependent steps, and both are now wrapped in their own error handling so a failure there can no longer block anything else. The Quick Descent click handler itself was also missing the same error handling Staged Descent’s already had - a runtime failure anywhere in the plan search previously failed completely silently, with no way to tell it had happened; it’s now caught and surfaced the same way.

Staged Descent’s Monte-Carlo area never actually being shown: found the real cause - `showStagedDescentPlan()` had no `map.fitBounds(...)` call anywhere in it, at all. The plan itself computed correctly (the side panel already showed concrete numbers - total time, ballast, expected landing location) and the landing-area polygon was drawn correctly, but the map never moved to actually show it. For a result landing well away from the clicked point (routine for staged descent, given its longer time horizon), that left the correctly-computed area sitting completely outside the visible viewport - which is very likely what looked like “the Monte-Carlo area doesn’t work”

despite the underlying computation being fine. Now zooms to the landing area immediately after it's computed, the same pattern used for Quick Descent's own result.

Staged Descent now also uses real ensemble data when available: previously only used the age-based heuristic scatter regardless of whether a commercial Open-Meteo key with ensemble access was configured - the only one of the three functions that didn't. Now checks for real ensemble members first (each sample drawing an actual member's own wind profile, genuine forecast uncertainty rather than a random perturbation) and only falls back to the heuristic when no ensemble data is available, matching Landing Area and Quick Descent's own behaviour.

Tower and telecom/antenna mast marker diameters doubled on the map (radius 1.3→2.6 and 1.6→3.2 respectively).

Quick Descent zoom, still under investigation: re-verified the entire chain (mode-switch state, the zoom call's position in the function, the click handler's error handling) line by line and found no further concrete bug - everything checked reads correctly. Added detailed console logging around the search result and the zoom call itself (the found delay, the resulting hull's bounds, current position) so the actual live behaviour can be read directly from the console on the next test, rather than continuing to theorize about it without being able to see what the search is actually finding.

GPS altitude vs. lat/lon: many devices - especially desktop/laptop computers without a real GPS chip, using WLAN/IP-based location instead - report a real latitude/longitude but never an altitude at all (`pos.coords.altitude` stays `null` indefinitely, not just on the first fix). The app already fell back to holding the last known altitude value in that case, which is correct behaviour, but gave no visible sign that this was happening - a fixed default value could look exactly like a stuck/broken GPS reading rather than the platform limitation it actually is. The "Alt AMSL" label now reads "Alt AMSL (no GPS altitude - held)" in amber whenever no real altitude reading has ever been received, reverting to the normal "Alt AMSL (GPS)" label the moment a device that does provide one is used.

Wind sounding panel

A detailed vertical wind profile, opened via the new button next to the E/H model badge in the header. Shows wind speed and direction from 0m AGL up to current altitude + 1750m (a fixed buffer, not a percentage), at either the current position or the currently active planned descent point (Quick Descent's own marker, or Staged Descent's - selectable via radio buttons, updates live while dragging either marker along the reference trajectory).

Deliberately shows the model's own real, raw pressure-level data points connected by straight lines - not a smoothed/densely-interpolated curve, which would imply more vertical resolution than the underlying forecast actually has.

The primary (currently auto-selected or manually chosen) model gets full meteorological wind barbs - shaft pointing toward where the wind is coming FROM (standard convention), full barb = 10kn, half barb = 5kn, pennant = 50kn - plus a degree label at each point. Additional models (only those with actual geographic coverage at the queried point are offered as toggles) can be overlaid as thin colored curves with simple arrows instead of full barbs - these arrows point the OPPOSITE way from the primary's barbs, toward where the wind is blowing TO, since that reads more intuitively for a quick visual comparison of several curves at once. An arrow's degree label gets a light red background (and the arrow itself turns red) whenever that model's direction at that altitude deviates by more than 25° from the primary model's own direction there.

A “ø Avg” toggle computes and overlays a genuine per-altitude weighted average across the primary plus whichever comparison models are currently selected - weighted by both grid resolution (finer wins) and model-run age (fresher wins), not an unweighted mean.

Y-axis is AMSL throughout; the intercept altitude (from Descent Parameters) is marked with a dashed line, and only the very bottom of the scale additionally shows the equivalent AGL value in parentheses, since AGL matters most near the ground. X-axis (wind speed in kn) is fully dynamic, scaling to whatever the actually-displayed data’s range is. A vertical dual-handle range slider next to the chart lets the shown altitude band be narrowed in on; using it for the first time also reveals a second, horizontal dual-handle slider below the chart for narrowing the wind-speed range shown on the x-axis (hidden until then, since it isn’t needed at the default full-range view).

Button icon is a bold royal-blue zigzag in a dark grey frame, next to the E/H model badge in the header - the third design iteration after two earlier, more literal attempts (a framed sounding trace, several composite scenes with balloons/thermometers/wind roses) turned out too visually busy to read clearly at toolbar size.

A third position option, “Marker location”, is available alongside “Current position” and “Planned descent point” - tapping anywhere on the map while the panel is open drops a distinctive royal-blue ring marker there (separate from every other marker type in the app) and switches to showing that point’s profile; the radio option stays disabled until a marker has actually been placed.

Comparison models get a heavier black outline drawn behind their curve (in addition to the toggle button’s own bold black border when active) so which models are currently selected reads clearly at a glance, both in the toggle row and in the chart itself.

Y-axis label overlap fixed (23.08.2026): AMSL and AGL numbers were sharing the same column and overlapping each other. Now in two separate columns - AGL only shown (and only up to the intercept altitude, per the earlier design decision) in the left column, AMSL throughout in a second column to its right. The intercept marking itself was moved from a text label at the chart’s right edge into a small “intcpt” label in the AGL column, with the dashed line across the chart unchanged. A distinct green ground line (true 0m AGL) was also added, separate from the intercept line.

Header button resized (23.08.2026): the button had grown large enough to push the header’s own model-name/cadence rows apart. Shrunk substantially (14px icon in a compact, low-contrast frame with a light grey fill, no separator line) so it fits within the existing row height instead of expanding it.

Speed-range slider repositioned: moved from below the whole chart into the chart’s own layout, directly under the x-axis tick numbers and above the “wind speed (kn)” axis title - implemented as an absolutely-positioned overlay on the chart, with the SVG’s own height increased to make room.

Avg toggle now matches the other model toggles’ on/off styling (thicker black border when active), and the primary model itself became toggleable too (default on) - its curve/barbs can be hidden the same way as any comparison model, though its underlying data stays available for the red-flag deviation check on comparison arrows regardless of whether it’s currently drawn.

Location radio buttons fit on one line: shortened labels (Current / Planned point / Marker) and reduced font size, since all three plus “Planned descent point”’s and “Marker location”’s full wording didn’t fit the panel’s 350px width on one row.

Wind sounding chart resized and reorganised (23.08.2026)

Plot area height increased 20% (230→276px), with the vertical zoom slider's own track grown proportionally (200→240px) so it still spans the full plot range. The horizontal speed-range slider was moved from mid-chart (where it collided with the "wind speed (kn)" axis title) to directly below the plot's x-axis line, roughly level with the vertical slider's own bottom label - tick numbers and the axis title now sit further down below both sliders, in a clear top-to-bottom order with no overlap. Gridline density increased from 5×4 to 9×6 divisions for finer visual reference.

Entire page below the header went blank - the actual root cause (23.08.2026)

Restructuring the header's model chip earlier the same day (to fix the wind-sounding button's sizing) introduced a genuine HTML bug: a new outer wrapper `<div>` was added around the chip and correctly closed, but the chip's own *original* closing `</div>` - now redundant - was left in place rather than removed. That stray extra `</div>` closed a much larger, unrelated ancestor container prematurely, which cascaded into the entire map/sidebar structure below it landing outside where it belonged in the DOM - rendering as a blank page below the header, exactly as reported and screenshotted.

Found via a proper tag-balance audit: counting every `<div>` against every `</div>` across the whole body (297 vs 298, a 1-tag imbalance), then a stack-based parser that walks the tags in order and reports the exact line where the stack empties out despite a closing tag still following - which pointed directly at the stray tag. Both checks now confirm exactly balanced (297/297, stack empties cleanly at the very end) and are being added to the standing deployment checklist as a required step whenever HTML structure (not just JS) is touched, alongside the syntax/TDZ/brace checks already in place - none of the earlier checks would have caught this, since it's a markup nesting problem, not a JavaScript one.

Monte-Carlo area investigation, done properly this time (23.08.2026)

Previous investigations of this recurring report only checked static structural integrity (function/marker/layer names still present, click-handler registration order) - never whether the code actually RUNS without error, since that requires a real JS engine executing it, not just parsing it. Set up a genuine runtime test this time: loaded the actual `index.html` in a real DOM (jsdom) with stubbed Leaflet/Chart.js/fetch, actually executed the boot sequence, then actively called `recomputeAll()` and simulated a full `planDescentToTarget()` call with a realistic mocked wind-forecast response.

Result: the core Monte-Carlo computation chain ran end-to-end without any error - the search found a result, the landing-area hull was computed (10-12 points), `state.plannedLandingActive` and `state.plannedLandingPolygon` were set correctly, and the zoom call fired as expected. This is a materially different (and much stronger) form of verification than anything done in earlier rounds - actual execution, not just inspection.

Given the core logic is now verified to run correctly, the two most plausible remaining explanations are things outside the JS logic itself:

1. **Browser HTTP caching of `index.html` itself** - no `Cache-Control`

meta tags were present before this change, meaning a browser could serve a stale cached copy of the whole page depending on the hosting server's own caching headers (which aren't controllable from here). Added `Cache-Control: no-cache, no-store, must-revalidate` plus `Pragma`/`Expires` meta tags so the browser is explicitly told never to cache the page, regardless of server headers.
2. The service worker (`sw.js`) was re-checked and confirmed to only cache map TILE requests, never `index.html` itself - ruled out as a cause.

If the report persists after this, the next most useful thing would be the actual browser console output at the moment it happens, since the runtime logic itself is now confirmed correct in isolation.

Cloudflare Worker proxy replaces the embedded raw GitHub token (24.08.2026)

Following up on the embedded-token discussion: the raw GitHub PAT was removed from both `index.html` and `admin.html`'s source entirely. A small Cloudflare Worker (`cloudflare-worker.js`, deployed separately at Cloudflare, not part of this repo's own static files) now sits between both apps and the GitHub Gist API - it holds the real token only as a server-side Secret (never sent to the browser), and exposes a narrow GET/PUT interface guarded by its own `APP_SECRET` header check. Both apps now call the worker's URL instead of `api.github.com` directly. The `WORKER_URL`/`WORKER_APP_SECRET` constants embedded in the source are still technically visible client-side the same way the old token was, but their blast radius is much smaller - even if leaked, they only grant read/write access to this one narrow store through this one worker, not the broader GitHub account a real PAT would expose (no ability to create arbitrary new gists, access other repos, etc.). `admin.html` still keeps a raw-GitHub-token path (its own separate, clearly-marked "setup only" section) purely for the one-time act of creating a brand new Gist, since the worker can only read/write an existing one, not create one.

A live-tested, hard lesson along the way: the first GitHub token shared for this setup was a **fine-grained** token (`github_pat_...` prefix) - those cannot access the Gist API at all (confirmed via a live HTTP 403 "Resource not accessible by personal access token"), regardless of what permissions are granted on it, since Gists sit outside fine-grained tokens' repository-scoped permission model entirely. A **classic** token (`ghp_...` prefix) with just the `gist` scope was needed instead, and worked immediately.

A second, subtler bug found once the worker itself was live (24.08.2026): the worker consistently returned HTTP 502 even with a fully correct token/Gist ID (verified by testing the exact same pair directly against GitHub's API via `curl`, which succeeded). The cause: GitHub's API rejects any request without a `User-Agent` header - `curl` supplies its own by default, which is why the direct test worked, but Cloudflare Workers' own `fetch()` does not add one automatically. The worker's own outbound request to GitHub was missing it entirely. Diagnosed via the worker's live Observability/Events log (Cloudflare dashboard → the worker → Observability tab), which showed the real request/response pair including the raw GitHub error text once the worker's own error response was extended to include it - the specific detail that made this identifiable rather than just "some 502 somewhere". Fixed by adding an explicit `User-Agent` header to every outbound GitHub call in `cloudflare-worker.js`.

Password max length increased 8→10 characters (24.08.2026), applied consistently in both `admin.html`'s user-editing table and `index.html`'s own login field - the two have to match, since a password created via the AdminApp that's longer than what the login field itself allows typing would be permanently impossible to actually log in with.

Login gate UI refined (24.08.2026): user-code field's placeholder changed "ABC"→"User Code", the flight-profile name field's placeholder changed "Name"→"Flightprofile name". The single combined login+open-profile action is now a button labeled "Open" (previously "Unlock", now positioned directly under the profile fields it also applies to), with "New profile" moved to its own row directly beneath it rather than sitting inline next to the name/PIN inputs. Along the way, a CSS specificity bug was caught before shipping: giving the "New profile" button the shared `.ghost` class wouldn't actually have applied that style, since `#passwordGate button`'s ID selector outranks a plain class selector - fixed by setting its secondary-button look via explicit inline styles instead.

User Code placeholder was visually truncated (24.08.2026): `text-transform:uppercase` on the field applies to its placeholder text too, rendering "User Code" as "USER CODE" - wider than the field's original width could actually display. Fixed by keeping the placeholder itself in normal mixed case (a CSS rule targeting `::placeholder` specifically, leaving actual typed input still auto-uppercased as before) plus a modest width increase as extra margin.

"New profile" at login never actually created anything - real bug, not just confusing UI (24.08.2026): the button cleared the name/PIN fields and simply logged in locally, without ever writing a new profile to the shared store - which is exactly why a "created" profile never showed up in the AdminApp or the footer afterward. Rebuilt as its own complete flow: validates the login credentials, requires both a name and PIN to actually be filled in, checks the name isn't already taken, then genuinely writes the new profile (with an empty settings object - nothing has been configured yet this early, before the app itself has even booted) to the shared store via the worker, and only then marks it as the active profile. Verified with a live-executed test (not just inspected) against a simulated worker response, confirming the actual PUT payload correctly included the new profile alongside the existing user table, and that the footer/Settings-panel both reflected the newly active profile immediately.

Sub-mode switch never actually moved the map - the real root cause (23.08.2026)

A recurring report ("zoom goes back to current position instead of the landing area when switching to Quick or Staged") turned out to have a genuinely different root cause than the earlier fixes to `planDescentToTarget` and `showStagedDescentPlan` - those only fire after a brand-new search computes a result and correctly zoom to it. `setSubMode()` - the function behind clicking the Quick/Staged tabs themselves - had no `fitBounds` call at all: switching back to a subfunction that already had a result sitting on the map (from earlier in the session) left the view wherever it happened to be, since nothing ever moved it there. Now reuses the exact same `fitBounds` logic the manual "center" buttons already used for each subfunction's own result, applied directly inside the tab-switch handler itself.

iPad-specific panel layout bugs (23.08.2026)

Staged Descent chart bleeding into the panel's header text: the chart's own SVG had `overflow:visible` (to avoid clipping labels near its edges), with only a 14px top margin inside its own coordinate space - on narrower viewports where the SVG scales down more aggressively (like an iPad), content sitting close to that margin could end up rendering outside the SVG's own box and visually overlapping the header text that precedes it in the panel, since nothing was clipping that overflow at the wrapper level. Fixed by giving the

wrapping `<div>` its own `overflow-y:hidden` (vertical overflow only, horizontal label overflow is unaffected) and increasing the chart's own top margin (14→22) for extra buffer.

Wind sounding panel not reaching the right screen edge: it previously used the same sidebar-anchored positioning as the Staged Descent panel (left edge = sidebar's left edge, width capped to whatever space remained to the screen's right edge) - on a narrower device this could squeeze the panel down to an unusably narrow width well before it would reach the actual screen edge. Given its own dedicated positioning function instead: always anchored to the screen's own right edge at its full 350px width (or slightly less only on very narrow screens), deliberately allowed to extend into the map area on the left when needed. The right-side floating map controls (base-layer picker, zoom buttons, scale bar, and the wind-particle height slider's own control stack) are shifted left by however far the panel actually overlaps them, via Leaflet's own `.leaflet-top/bottom.leaflet-right` containers plus the app's own `.map-controls-stack`, rather than letting the panel sit on top of and hide them - reset again the moment the panel closes, and recalculated on window resize (e.g. rotating the iPad).

Wind sounding marker was leaking into Quick/Staged Descent click handling (23.08.2026)

The map has several independent `click` listeners registered (Leaflet calls every one of them on a single click, not just the "most relevant" one) - the wind-sounding panel's own marker-placement handler was one of them, alongside the main handler that drives Quick/Staged Descent's own click logic (placing the Staged marker, setting a Quick Descent target, test-mode repositioning). With the wind-sounding panel open, a tap meant only for its own marker was ALSO reaching that main handler, which could silently place or recompute a Staged Descent plan from a click the person never intended for it. Fixed with an early-return guard at the very top of the main click handler: while the wind-sounding panel is visible, it does nothing at all, leaving that click exclusively to the wind-sounding marker's own handler.

This very plausibly also explains the separately-reported "Quick/Staged Descent zoom doesn't go to the Monte-Carlo area" - both `planDescentToTarget` and `showStagedDescentPlan` were re-verified (again) to have their `fitBounds` calls correctly placed with nothing overriding them afterward, so a genuinely unrelated click accidentally triggering an unwanted Staged Descent computation (with its own, unexpected zoom) is a much better explanation for what looked like "zoom doesn't work" than a bug in the zoom logic itself, which has now been checked this thoroughly multiple times without finding a fault.

Wind sounding panel refinements (23.08.2026)

Header button: restructured so it sits beside the whole two-row model chip (as a flex sibling) rather than nested inside the chip's first row - this lets it be sized independently (now 22px icon) without stretching the chip's own row height and pushing its second line down, which is what made it look oversized and gap-inducing in an earlier attempt.

Vertical range slider restyled to match the thin-line horizontal slider (a 2px line rather than a 16px filled bar) for visual consistency. The horizontal speed-range slider now also shows its current min/max values as text labels at each end, matching the vertical slider's own top/bottom labels.

Reset-to-original-view button: appears (bottom-right, below the vertical slider, at the horizontal slider's height) the first time the height slider is actually used, resetting both the altitude and speed ranges back to their full defaults - the horizontal speed slider itself stays visible after a reset rather than hiding again, per explicit instruction.

Marker cleanup on close: previously only removed when re-clicking the toggle button to close the panel - now also removed via the panel's own "X" close button, via a shared cleanup function so both paths can't drift apart again.

Avg toggle now uses the exact same on/off styling (thicker black border when active) as the model toggle buttons, and the **primary model itself became toggleable** too (default on) - hiding it only affects what's drawn, its data still backs the red-flag deviation check on comparison arrows regardless.

Location radio buttons shortened (Current / Planned point / Marker) to fit on one line within the panel's width.

Telecom/antenna mast markers made bolder (23.08.2026)

Radius increased (3.2→5) and given a dark contrasting outline (weight 0.5→1.6) instead of the previous barely-visible thin edge, so they read clearly against the map at typical zoom levels.

Wind sounding panel - map click handler isolation

The wind-sounding panel's own map click handler (for placing the "Marker location" point) is registered strictly AFTER the main descent-planning click handler in the script, and Leaflet calls listeners for the same event in registration order - so it cannot block or interfere with that handler's own execution. It's additionally wrapped in its own try/catch regardless, so a failure inside it can never propagate anywhere else. A Monte-Carlo-area regression report investigated on 23.08.2026 could not be traced to any structural change in this feature or any other recent edit - all the core landing-area functions and their marker/layer references were re-verified intact.

Known limitations

- Adiabatic braking and Monte-Carlo scatter (without a commercial API key/real ensemble data) are approximations, not calibrated against real flight data - a planning aid, not a certified instrument.
- The landing-point terrain check is an approximation, not a true line-of-sight/obstacle-clearance calculation - ring-sampled elevation plus OSM tags, since actual obstacle heights aren't available from any free source used here.
- aprs.fi is confirmed non-functional (browser CORS restriction on their end, not fixable from this app).
- MeteoGate/E-SOH per-station value parsing is implemented but not yet confirmed against a live response.
- This is a static file with no server: after any change, it has to be re-uploaded to wherever it's hosted before the live site reflects it. On iOS specifically, both the page itself and the service worker can be cached persistently enough that a hard reload or removing/re-adding the home-screen icon may be needed to pick up a new version.
- Cloud file pickers (Dropbox, Google Drive, etc.) shown when uploading a file are controlled by iOS/the browser based on

installed provider apps, not by this page.